

UNIVERSIDADE FEDERAL DO RIO GRANDE — FURG

Centro de Ciências Computacionais — C3

Sistemas para Internet 2 — Turma U — 1º Semestre de 2026

Nomes: Luis Henrique K. Reichow - 162589 e Matheus N. Buttow -164400

RELATÓRIO TÉCNICO

Web Proxy com Controle de Conteúdo

Prof. Dr. André Prisco Vargas

Rio Grande, 2026

1. Justificativa da Tecnologia Escolhida

Para a implementação do proxy HTTP, a dupla optou pela linguagem Python com o microframework Flask. Essa escolha se justifica por múltiplos fatores técnicos e práticos.

Em primeiro lugar, Flask oferece uma abstração de alto nível sobre o protocolo HTTP, permitindo que a lógica de roteamento e processamento de requisições seja implementada de forma clara e concisa. Isso reduziu o tempo de desenvolvimento e facilitou a leitura e manutenção do código.

Em segundo lugar, a biblioteca requests do Python simplifica enormemente o envio de requisições ao servidor de origem, oferecendo suporte nativo a todos os métodos HTTP (GET, POST, PUT, DELETE, PATCH), redirecionamentos e personalização de cabeçalhos.

Comparado à alternativa de sockets TCP puros, o Flask elimina a necessidade de implementar manualmente o parsing do protocolo HTTP — tarefa complexa e propensa a erros. Em relação a tecnologias como Node.js ou Spring, a escolha pelo Python reflete a familiaridade da dupla com o ecossistema e o suporte técnico garantido pelo professor para essa stack.

1.1 Vantagens Percebidas

- Código limpo e legível, com roteamento declarativo via decoradores Flask
- Biblioteca requests robusta para chamadas HTTP ao servidor de origem
- Facilidade na manipulação de strings e expressões regulares para o filtro de conteúdo
- Excelente suporte a JSON nativo (módulo json da biblioteca padrão)

1.2 Dificuldades Encontradas

- Distinção entre requisições originadas pelo navegador e pelo cURL exigiu tratamento especial do campo Host
- Passagem de cabeçalhos sem modificar o Content-Length gerou erros em algumas requisições
- Respostas binárias (imagens, PDFs) retornadas como texto causaram problemas iniciais de encoding

2. Decisões de Implementação

2.1 Rota Genérica com path:url

A rota principal do proxy captura qualquer caminho via o padrão `/<path:url>`, permitindo que tanto URLs completas (`http://...`) quanto caminhos relativos sejam interceptados. Isso foi necessário para compatibilidade com dois modos de uso: navegador e cURL.

2.2 Tratamento Diferenciado: Navegador vs. cURL

Ao usar o cURL com a flag `-x` (proxy), o cliente envia a URL completa no cabeçalho Host. O Flask captura isso em `request.host`, que inclui o domínio alvo. Já no caso do navegador, o host aponta para `localhost:5000` e a URL completa vem no caminho da requisição. Essa distinção exigiu uma lógica condicional explícita na função principal do proxy.

2.3 Limpeza de Cabeçalhos

Os cabeçalhos Host e Content-Length são removidos antes de repassar a requisição ao servidor de origem. O Host é removido para que o servidor alvo não receba localhost:5000 como destino. O Content-Length é removido porque a biblioteca requests recalcula automaticamente esse valor, evitando conflitos.

2.4 Filtro Case-Insensitive

A substituição de palavras utiliza expressões regulares com a flag re.IGNORECASE, garantindo que variações como 'Merda', 'MERDA' e 'merda' sejam todas substituídas. O mapa de substituições é lido dinamicamente do arquivo words.json a cada requisição, permitindo atualizações sem reiniciar o servidor.

2.5 Log de Acessos

Cada requisição é registrada em log.txt com timestamp, URL e ação executada (permitido, filtrado ou bloqueado). A ação é determinada comparando o conteúdo original com o conteúdo processado: se forem diferentes, houve filtragem.

3. HTTPS e o Filtro de Conteúdo

3.1 Por que o filtro não funciona em sites HTTPS?

O protocolo HTTPS (HTTP Secure) adiciona uma camada de criptografia TLS/SSL entre o cliente e o servidor. Isso significa que todo o conteúdo — incluindo cabeçalhos e corpo da resposta — é cifrado de ponta a ponta.

Quando um cliente tenta acessar um site HTTPS através do proxy, ele utiliza o método HTTP CONNECT para estabelecer um túnel TCP. O proxy apenas encaminha os bytes cifrados entre o cliente e o servidor, sem conseguir inspecionar nem modificar o conteúdo, pois não possui as chaves criptográficas necessárias para a decifração.

Portanto, qualquer tentativa de aplicar o filtro de palavras sobre tráfego HTTPS resultaria apenas em bytes ininteligíveis. O proxy atual não suporta HTTPS, retornando erro para esse tipo de requisição.

3.2 O que seria necessário para funcionar?

Para inspecionar e modificar tráfego HTTPS, seria necessário implementar um proxy SSL interceptor (também chamado de man-in-the-middle legítimo). Isso envolveria:

- Geração de uma Autoridade Certificadora (CA) própria, instalada nos clientes como confiável
- Ao receber uma conexão HTTPS, o proxy estabelece um certificado falso assinado pela CA local
- O proxy decifra o tráfego, aplica os filtros, e re-cifra antes de encaminhar ao servidor real
- Ferramentas como mitmproxy implementam exatamente essa abordagem

Essa solução tem implicações éticas e legais significativas, já que rompe o modelo de confiança do HTTPS. Em ambientes corporativos, é utilizada com consentimento explícito dos usuários e certificados instalados administrativamente.

4. O que Aprendemos sobre o Protocolo HTTP

A implementação do proxy proporcionou um contato direto e prático com aspectos do protocolo HTTP que permaneceriam abstratos numa abordagem puramente teórica.

- Estrutura real das requisições: compreendemos na prática o papel dos cabeçalhos como Host, Content-Type, Content-Length e como eles são interpretados por servidores e proxies.
- Modelo cliente-servidor intermediado: entendemos como um proxy atua como servidor para o cliente e como cliente para o servidor de origem, duplicando seu papel na comunicação.
- Diferença entre HTTP e HTTPS: a implementação tornou concreta a compreensão de por que o HTTPS impede a inspeção de conteúdo por terceiros — incluindo proxies.
- Método CONNECT: aprendemos que navegadores usam esse método especial para solicitar ao proxy que estabeleça um túnel TCP direto para sites HTTPS.
- Redirecionamentos HTTP (301/302): percebemos que o comportamento padrão da biblioteca requests de seguir redirecionamentos automaticamente precisava ser desativado (`allow_redirects=False`) para que o cliente recebesse a resposta original e pudesse decidir como proceder.

5. Uso de Inteligência Artificial

A dupla utilizou o assistente Claude (Anthropic) como ferramenta de apoio ao longo do desenvolvimento.

5.1 Como foi utilizado

- Geração do README.md do repositório, posteriormente revisado pela dupla
- Esclarecimento de dúvidas sobre o comportamento da biblioteca Flask e requests
- Geração deste relatório técnico, com base no código desenvolvido pela dupla e no enunciado do trabalho
- Revisão de trechos de código para identificar possíveis bugs

5.2 O que foi modificado

Todo o código-fonte do proxy foi escrito e compreendido pela dupla. A IA foi utilizada principalmente para documentação e esclarecimento de conceitos, não para geração direta de código funcional. Os textos gerados pela IA foram revisados e adaptados para refletir com precisão as decisões técnicas efetivamente tomadas.

5.3 O que aprendemos com a interação

A interação com a IA foi útil para estruturar o raciocínio sobre problemas como o tratamento de HTTPS e a distinção entre requisições de navegador e cURL. As respostas geradas serviram como ponto de partida para investigação, sendo sempre verificadas e complementadas com a documentação oficial e testes práticos.